

tf.GradientTape



View source on GitHub (<https://github.com/tensorflow/tensorflow/blob/v2.16.1/tensorflow/python/eager>)

Record operations for automatic differentiation.



View aliases

Main aliases

`tf.autodiff.GradientTape` (https://www.tensorflow.org/api_docs/python/tf/GradientTape)

Compat aliases for migration

See [Migration guide](https://www.tensorflow.org/guide/migrate) (<https://www.tensorflow.org/guide/migrate>) for more details.

`tf.compat.v1.GradientTape` (https://www.tensorflow.org/api_docs/python/tf/GradientTape)

```
tf.GradientTape(  
    persistent=False, watch_accessed_variables=True  
)
```

Used in the notebooks

Used in the guide	Used in the tutorials
• Introduction to gradients and automatic differentiation (https://www.tensorflow.org/guide/autodiff) • Advanced automatic differentiation (https://www.tensorflow.org/guide/advanced_autodiff) • Better performance with tf.function (https://www.tensorflow.org/guide/function)	• Deep Convolutional Generative Adversarial Network (https://www.tensorflow.org/tutorials/generative/dcgan) • DeepDream (https://www.tensorflow.org/tutorials/generative/deepdream) • pix2pix: Image-to-image translation with a conditional GAN

Used in the guide	Used in the tutorials
• TensorFlow basics (https://www.tensorflow.org/guide/basics) • Effective Tensorflow 2 (https://www.tensorflow.org/guide/effective_tf2)	(https://www.tensorflow.org/tutorials/generative/pix2pix) • Neural style transfer (https://www.tensorflow.org/tutorials/generative/style_transfer) • Custom training: walkthrough (https://www.tensorflow.org/tutorials/customization/custom_training_walkthrough)

Operations are recorded if they are executed within this context manager and at least one of their inputs is being "watched".

Trainable variables (created by `tf.Variable`

(https://www.tensorflow.org/api_docs/python/tf/Variable) or `tf.compat.v1.get_variable` (https://www.tensorflow.org/api_docs/python/tf/compat/v1/get_variable), where `trainable=True` is default in both cases) are automatically watched. Tensors can be manually watched by invoking the `watch` method on this context manager.

For example, consider the function $y = x * x$. The gradient at $x = 3.0$ can be computed as:

```
>>> x = tf.constant(3.0)
>>> with tf.GradientTape() as g:
...     g.watch(x)
...     y = x * x
>>> dy_dx = g.gradient(y, x)
>>> print(dy_dx)
tf.Tensor(6.0, shape=(), dtype=float32)
```

GradientTapes can be nested to compute higher-order derivatives. For example,

```
>>> x = tf.constant(5.0)
>>> with tf.GradientTape() as g:
...     g.watch(x)
...     with tf.GradientTape() as gg:
...         gg.watch(x)
...         y = x * x
```

```

...    dy_dx = gg.gradient(y, x) # dy_dx = 2 * x
>>> d2y_dx2 = g.gradient(dy_dx, x) # d2y_dx2 = 2
>>> print(dy_dx)
tf.Tensor(10.0, shape=(), dtype=float32)
>>> print(d2y_dx2)
tf.Tensor(2.0, shape=(), dtype=float32)

```

By default, the resources held by a GradientTape are released as soon as GradientTape.gradient() method is called. To compute multiple gradients over the same computation, create a persistent gradient tape. This allows multiple calls to the gradient() method as resources are released when the tape object is garbage collected. For example:

```

>>> x = tf.constant(3.0)
>>> with tf.GradientTape(persistent=True) as g:
...     g.watch(x)
...     y = x * x
...     z = y * y
>>> dz_dx = g.gradient(z, x) # (4*x^3 at x = 3)
>>> print(dz_dx)
tf.Tensor(108.0, shape=(), dtype=float32)
>>> dy_dx = g.gradient(y, x)
>>> print(dy_dx)
tf.Tensor(6.0, shape=(), dtype=float32)

```

By default GradientTape will automatically watch any trainable variables that are accessed inside the context. If you want fine grained control over which variables are watched you can disable automatic tracking by passing watch_accessed_variables=False to the tape constructor:

```

>>> x = tf.Variable(2.0)
>>> w = tf.Variable(5.0)
>>> with tf.GradientTape(
...     watch_accessed_variables=False, persistent=True) as tape:
...     tape.watch(x)
...     y = x ** 2 # Gradients will be available for `x`.
...     z = w ** 3 # No gradients will be available as `w` isn't being watched.
>>> dy_dx = tape.gradient(y, x)
>>> print(dy_dx)
tf.Tensor(4.0, shape=(), dtype=float32)

```

```
>>> # No gradients will be available as `w` isn't being watched.
>>> dz_dw = tape.gradient(z, w)
>>> print(dz_dw)
None
```

Note that when using models you should ensure that your variables exist when using `watch_accessed_variables=False`. Otherwise it's quite easy to make your first iteration not have any gradients:

```
a = tf.keras.layers.Dense(32)
b = tf.keras.layers.Dense(32)

with tf.GradientTape(watch_accessed_variables=False) as tape:
    tape.watch(a.variables) # Since `a.build` has not been called at this point
                           # `a.variables` will return an empty list and the
                           # tape will not be watching anything.

    result = b(a(inputs))
    tape.gradient(result, a.variables) # The result of this computation will be
                                       # a list of `None`s since a's variables
                                       # are not being watched.
```

Note that only tensors with real or complex dtypes are differentiable.

Args

persistent	Boolean controlling whether a persistent gradient tape is created. False by default, which means at most one call can be made to the <code>gradient()</code> method on this object.
watch_accessed_variables	Boolean controlling whether the tape will automatically watch any (trainable) variables accessed while the tape is active. Defaults to True meaning gradients can be requested from any result computed in the tape derived from reading a trainable Variable . If False users must explicitly watch any Variables they want to request gradients from.

Methods

batch_jacobian

View source

(<https://github.com/tensorflow/tensorflow/blob/v2.16.1/tensorflow/python/eager/backprop.py#L1198-L1345>)

```
batch_jacobian(  
    target,  
    source,  
    unconnected_gradients=tf.UnconnectedGradients.NONE (https://www.tensorflow.org/api\_docs/python/tf/UnconnectedGradients),  
    parallel_iterations=None,  
    experimental_use_pfor=True  
)
```

Computes and stacks per-example jacobians.

See [wikipedia article](http://en.wikipedia.org/wiki/Jacobian_matrix_and_determinant) (http://en.wikipedia.org/wiki/Jacobian_matrix_and_determinant) for the definition of a Jacobian. This function is essentially an efficient implementation of the following:

```
tf.stack([self.jacobian(y[i], x[i]) for i in range(x.shape[0])]).
```

Note that compared to [`GradientTape.jacobian`](https://www.tensorflow.org/api_docs/python/tf/GradientTape#jacobian)

(https://www.tensorflow.org/api_docs/python/tf/GradientTape#jacobian) which computes gradient of each output value w.r.t each input value, this function is useful when `target[i, ...]` is independent of `source[j, ...]` for $j \neq i$. This assumption allows more efficient computation as compared to [`GradientTape.jacobian`](https://www.tensorflow.org/api_docs/python/tf/GradientTape#jacobian)

(https://www.tensorflow.org/api_docs/python/tf/GradientTape#jacobian). The output, as well as intermediate activations, are lower dimensional and avoid a bunch of redundant zeros which would result in the jacobian computation given the independence assumption.

Note: Unless you set `persistent=True` a GradientTape can only be used to compute one set of gradients (or jacobians).

Note: By default the `batch_jacobian` implementation uses parallel for (pfor), which creates a `tf.function` under the hood for each `batch_jacobian` call. For better performance, and to avoid recompilation and vectorization rewrites on each call, enclose GradientTape code in `@tf.function`.

Example usage:

```

with tf.GradientTape() as g:
    x = tf.constant([[1., 2.], [3., 4.]], dtype=tf.float32)
    g.watch(x)
    y = x * x
batch_jacobian = g.batch_jacobian(y, x)
# batch_jacobian is [[[2, 0], [0, 4]], [[6, 0], [0, 8]]]

```

Args

target	A tensor with rank 2 or higher and with shape [b, y1, ..., y_n]. target[i, ...] should only depend on source[i, ...] .
source	A tensor with rank 2 or higher and with shape [b, x1, ..., x_m].
unconnected_gradients	a value which can either hold 'none' or 'zero' and alters the value which will be returned if the target and sources are unconnected. The possible values and effects are detailed in 'UnconnectedGradients' and it defaults to 'none'.
parallel_iterations	A knob to control how many iterations are dispatched in parallel. This knob can be used to control the total memory usage.
experimental_use_pfor	If true, uses pfor for computing the Jacobian. Else uses a tf.while_loop.

Returns

A tensor **t** with shape [b, y_1, ..., y_n, x1, ..., x_m] where **t[i, ...]** is the jacobian of **target[i, ...]** w.r.t. **source[i, ...]**, i.e. stacked per-example jacobians.

Raises

RuntimeError	If called on a used, non-persistent tape.
RuntimeError	If called on a non-persistent tape with eager execution enabled and without enabling experimental_use_pfor.
ValueError	If vectorization of jacobian computation fails or if first dimension of target and source do not match.

gradient

[View source](#)

(<https://github.com/tensorflow/tensorflow/blob/v2.16.1/tensorflow/python/eager/backprop.py#L960-L1083>)

```
gradient(  
    target,  
    sources,  
    output_gradients=None,  
    unconnected_gradients=tf.UnconnectedGradients.NONE (https://www.tensorflow.org/api\_d  
)
```

Computes the gradient using operations recorded in context of this tape.

Note: Unless you set **persistent=True** a GradientTape can only be used to compute one set of gradients (or jacobians).

In addition to Tensors, gradient also supports RaggedTensors. For example,

```
>>> x = tf.ragged.constant([[1.0, 2.0], [3.0]])  
>>> with tf.GradientTape() as g:  
...     g.watch(x)  
...     y = x * x  
>>> g.gradient(y, x)  
<tf.RaggedTensor [[2.0, 4.0], [6.0]]>
```

Args

target	a list or nested structure of Tensors or Variables or CompositeTensors to be differentiated.
sources	a list or nested structure of Tensors or Variables or CompositeTensors. target will be differentiated against elements in sources .
output_gradients	a list of gradients, one for each differentiable element of target. Defaults to None.
unconnected_gradients	a value which can either hold 'none' or 'zero' and alters the value which will be returned if the target and sources are unconnected. The possible

values and effects are detailed in 'UnconnectedGradients' and it defaults to 'none'.

Returns

a list or nested structure of Tensors (or IndexedSlices, or None, or CompositeTensor), one for each element in **sources**. Returned structure is the same as the structure of **sources**.

Raises

RuntimeError	If called on a used, non-persistent tape.
RuntimeError	If called inside the context of the tape.
TypeError	If the target is a None object.
ValueError	If the target is a variable or if unconnected gradients is called with an unknown value.

jacobian

[View source](#)

(<https://github.com/tensorflow/tensorflow/blob/v2.16.1/tensorflow/python/eager/backprop.py#L1085-L1196>)

```
jacobian(  
    target,  
    sources,  
    unconnected_gradients=tf.UnconnectedGradients.NONE (https://www.tensorflow.org/api\_docs/python/tf/UnconnectedGradients),  
    parallel_iterations=None,  
    experimental_use_pfor=True  
)
```

Computes the jacobian using operations recorded in context of this tape.

Note: Unless you set **persistent=True** a GradientTape can only be used to compute one set of gradients (or jacobians).

Note: By default the jacobian implementation uses parallel for (pfor), which creates a tf.function under the hood for each jacobian call. For better performance, and to avoid recompilation and vectorization rewrites on each call, enclose GradientTape code in @tf.function.

See [wikipedia article](http://en.wikipedia.org/wiki/jacobian_matrix_and_determinant) (http://en.wikipedia.org/wiki/jacobian_matrix_and_determinant) for the definition of a Jacobian.

Example usage:

```
with tf.GradientTape() as g:
    x = tf.constant([1.0, 2.0])
    g.watch(x)
    y = x * x
jacobian = g.jacobian(y, x)
# jacobian value is [[2., 0.], [0., 4.]]
```

Args

target	Tensor to be differentiated.
sources	a list or nested structure of Tensors or Variables. target will be differentiated against elements in sources .
unconnected_gradients	a value which can either hold 'none' or 'zero' and alters the value which will be returned if the target and sources are unconnected. The possible values and effects are detailed in 'UnconnectedGradients' and it defaults to 'none'.
parallel_iterations	A knob to control how many iterations are dispatched in parallel. This knob can be used to control the total memory usage.
experimental_use_pfor	If true, vectorizes the jacobian computation. Else falls back to a sequential while_loop. Vectorization can sometimes fail or lead to excessive memory usage. This option can be used to disable vectorization in such cases.

Returns

A list or nested structure of Tensors (or None), one for each element in **sources**. Returned structure is the same as the structure of **sources**. Note if any gradient is sparse (IndexedSlices), jacobian function currently makes it dense and returns a Tensor instead. This may change in the future.

Raises

RuntimeError	If called on a used, non-persistent tape.
RuntimeError	If called on a non-persistent tape with eager execution enabled and without enabling <code>experimental_use_pfor</code> .
ValueError	If vectorization of jacobian computation fails.

reset

View source

(<https://github.com/tensorflow/tensorflow/blob/v2.16.1/tensorflow/python/eager/backprop.py#L918-L952>)

`reset()`

Clears all information stored in this tape.

Equivalent to exiting and reentering the tape context manager with a new tape. For example, the two following code blocks are equivalent:

```
with tf.GradientTape() as t:
    loss = loss_fn()
with tf.GradientTape() as t:
    loss += other_loss_fn()
t.gradient(loss, ...) # Only differentiates other_loss_fn, not loss_fn
```

```
# The following is equivalent to the above
with tf.GradientTape() as t:
    loss = loss_fn()
    t.reset()
    loss += other_loss_fn()
```

```
t.gradient(loss, ...) # Only differentiates other_loss_fn, not loss_fn
```

This is useful if you don't want to exit the context manager for the tape, or can't because the desired reset point is inside a control flow construct:

```
with tf.GradientTape() as t:
    loss = ...
    if loss > k:
        t.reset()
```

stop_recording

View source

(<https://github.com/tensorflow/tensorflow/blob/v2.16.1/tensorflow/python/eager/backprop.py#L886-L916>)

```
@tf_contextlib.contextmanager
stop_recording()
```

Temporarily stops recording operations on this tape.

Operations executed while this context manager is active will not be recorded on the tape. This is useful for reducing the memory used by tracing all computations.

For example:

```
>>> x = tf.constant(4.0)
>>> with tf.GradientTape() as tape:
...     with tape.stop_recording():
...         y = x ** 2
>>> dy_dx = tape.gradient(y, x)
>>> print(dy_dx)
None
```

Yields

None

Raises

RuntimeError if the tape is not currently recording.

watch

[View source](#)

(<https://github.com/tensorflow/tensorflow/blob/v2.16.1/tensorflow/python/eager/backprop.py#L864-L884>)

```
watch(  
    tensor  
)
```

Ensures that `tensor` is being traced by this tape.

Args

tensor a Tensor/Variable or list of Tensors/Variables.

Raises

ValueError if it encounters something that is not a tensor.

watched_variables

[View source](#)

(<https://github.com/tensorflow/tensorflow/blob/v2.16.1/tensorflow/python/eager/backprop.py#L954-L958>)

```
watched_variables()
```

Returns variables watched by this tape in order of construction.

`__enter__`

View source

(<https://github.com/tensorflow/tensorflow/blob/v2.16.1/tensorflow/python/eager/backprop.py#L821-L824>)

```
__enter__()
```

Enters a context inside which operations are recorded on this tape.

`__exit__`

View source

(<https://github.com/tensorflow/tensorflow/blob/v2.16.1/tensorflow/python/eager/backprop.py#L826-L829>)

```
__exit__(  
    typ, value, traceback  
)
```

Exits the recording context, no further operations are traced.

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates. Some content is licensed under the [numpy license](https://numpy.org/doc/stable/license.html) (<https://numpy.org/doc/stable/license.html>).

Last updated 2024-04-26 UTC.